
L4-P02 练习手册

微调数据处理方法

涵盖实验：文本转微调数据 · 格式转换 · 流水线演示 · 去重清洗 · 中文数据清洗

共 5 个实验模块

实验 1：从自由文本生成微调数据

使用框架	Python 标准库，可选接入 OpenAI API
安装依赖	基础运行：无需额外依赖。使用 --use_llm 时：pip install openai tiktoken

代码文件

- 主程序：text_to_finetune.py
 简化版：simple_text_to_finetune.py
- 运行脚本：run_demo.bat / run_demo.sh

输入 / 输出

- 输入：普通文本、单个文档或文档目录
- 输出目录：output/
- 输出内容：
 - finetune_data.jsonl
 - *_with_metadata.json
 - *_report.json

程序流程

步骤	说明
1	导入与初始化：加载 json、re、argparse 等基础库，并初始化配置对象与处理流水线。
2	读取输入：接收单个文件、目录或内置示例文本作为原始语料输入。
3	文档解析：根据文件类型读取 txt、md、html、pdf、docx 等文本内容。
4	文本清洗：去除多余空白、异常换行和无效字符，得到较规范的正文文本。
5	文本切片：按照段落结构和长度限制，把长文本拆分成多个可处理片段。
6	知识点提取：从每个片段中抽取概念、流程、因果、对比等知识点。
7	QA 生成：基于知识点生成问答对，可选择规则生成或调用 LLM 生成。
8	质量过滤：删除指令过短、回答过短、模型自我引用等低质量样本。
9	去重处理：对生成结果执行哈希去重和相似性过滤。

10

导出结果：输出 Alpaca 风格微调数据，并同时生成元数据文件和处理报告。

适用场景：将教材、说明文档、技术资料等自由文本快速转成可用于 SFT 微调的数据。

实验 2：数据格式转换

使用框架

Python 标准库

安装依赖

无需额外依赖

支持的数据格式

- alpaca
- chatml
- sharegpt
- prompt_completion

代码文件

- 主程序：data_format_converter.py
- 运行脚本：run_demo.bat / run_demo.sh

输入 / 输出

- 输入：.json 或 .jsonl 格式的数据文件
- 输出目录：output/
- 输出文件命名规则：原文件名_output_目标格式.xxx

程序流程

步骤	说明
1	导入库：加载 argparse、json 和 pathlib，用于命令行解析、文件读写和路径处理。
2	读取输入文件：支持 .jsonl 逐行读取，也支持 .json 整体读取。
3	识别输入格式：根据字段自动判断数据属于 alpaca、chatml、sharegpt 还是 prompt_completion。
4	标准化消息结构：先把不同源格式统一转换为内部使用的 messages 表示。

5	处理上下文信息：在需要转成单轮格式时，把系统提示和历史对话折叠进 input 或 prompt。
6	导出目标格式：再从统一的 messages 表示转换成目标格式结构。
7	生成输出路径：按照'原文件名 + 输出格式'的规则生成输出文件名，并定位到 output 目录。
8	写回结果文件：根据后缀把结果保存为 .json 或 .jsonl 文件。
适用场景：整理不同来源的数据集格式，便于后续训练、评测和实验对比。	

实验 3：微调数据处理流水线演示

使用框架	Python 标准库
安装依赖	无需额外依赖

代码文件

- 主程序：data-pipeline-demo.py
 简化版：simple_pipeline.py
- 运行脚本：run_demo.bat / run_demo.sh

输入 / 输出

- 输入：内置脏数据样例或自定义 json 文件
- 输出目录：output/
- 输出内容：
 - demo_output.jsonl
 - demo_output_full.json
 - sample_input.json

程序流程

步骤	说明
1	加载输入数据：使用内置脏数据样例，或者读取用户传入的自定义 json 文件。
2	格式统一：识别 Alpaca、ShareGPT、多轮对话等格式，并统一成标准结构。

3	格式校验：检查必填字段、文本长度和特殊 token 等格式问题。
4	质量过滤：识别套话、AI 自我引用、回答冗长等低质量内容。
5	安全过滤：检测 PII、Prompt Injection 等安全与合规风险。
6	去重处理：执行精确去重和语义相似度去重。
7	多样性分析：统计指令长度、回答长度和高频关键词等信息。
8	导出结果：输出训练可用的 jsonl 文件和带元数据的完整 json 文件。

运行方式

```
python data-pipeline-demo.py
python data-pipeline-demo.py --auto
python data-pipeline-demo.py --input mydata.json
python data-pipeline-demo.py --save_input
```

适用场景：用于教学演示和理解微调数据清洗流水线的完整过程。

实验 4：语料去重与相似度过滤

使用框架	Sentence-Transformers + FAISS
安装依赖	pip install faiss-cpu numpy==1.26.4 tqdm \ huggingface_hub==0.23.4 \ transformers==4.41.2 \ accelerate==0.30.1 \ sentence-transformers==2.7.0
依赖模型	bge-small-zh-v1.5

代码文件

- 主程序：remove_dup_similar_text.py
- 模型下载脚本：model_downloader.py
- 运行脚本：run_demo.sh

输入 / 输出

- 输入文件：longtext_dialogues_chatml.jsonl

- 输出文件: cleaned_dialogues.jsonl
- 相似度阈值: 0.88 (越低越严格, 可能删除更多相似样本)

程序流程

步骤	说明
1	导入依赖: 加载 json、hashlib、faiss、numpy 和 sentence_transformers。
2	读取输入数据: 逐行加载 ChatML 格式的多轮对话样本。
3	拼接文本内容: 把每条样本中的多条 messages 内容合并成一段文本。
4	哈希去重: 对拼接后的文本计算 MD5, 删除完全重复样本。
5	加载句向量模型: 使用 paraphrase-multilingual-MiniLM-L12-v2 对文本进行编码。
6	建立 FAISS 索引: 将全部句向量加入相似度检索索引。
7	相似度过滤: 检索高相似样本并删除语义重复数据 (余弦相似度 $\geq$ 阈值时只保留一条)。
8	重新编号: 为保留下来的样本重新分配连续 id。
9	写出结果: 输出清洗后的 jsonl 文件。
适用场景: 清理对话类训练数据中的完全重复和语义重复样本, 降低训练集冗余。	

实验 5：数据质量与安全清洗

使用框架	Transformers + PyTorch
安装依赖	pip install torch transformers tqdm
依赖模型	Qwen1.5-1.8B / bert-multilingual-toxicity-classifier / pai-bert-base-zh-llm-risk-detection

代码文件

- 主程序: filter_chinese_dataset_offline.py
- 模型下载脚本: model_downloader.py
- 运行脚本: run_demo.sh

输入 / 输出

- 输入文件：raw_chinese_data.jsonl
- 输出文件：cleaned_chinese_data_offline.jsonl

程序总体逻辑

该程序在完全离线环境中运行，自动对中文语料进行三重质量检测与过滤，确保输出数据安全、自然、可用于大模型训练。

三重过滤模块说明

① 规则过滤 (rule_filter)

目的：先剔除明显不合格或无意义的文本。

- 是否为空或全是空格
- 长度是否在规定范围 (5 ~ 500 字)
- 是否包含网址 (http / www)
- 是否包含邮箱 (@)
- 是否包含 7 位以上连续数字 (可能是身份证或手机号)

若符合任一条件 → 直接丢弃该样本。

② 有害内容检测 (ToxicityDetector)

目的：用本地安全模型识别辱骂、色情、政治敏感、仇恨等内容。

- 模型 1：bert-multilingual-toxicity-classifier (多语言 TextDetox)
- 模型 2：pai-bert-base-zh-llm-risk-detection (阿里中文安全模型)

两个模型均输出安全 (0) / 有害 (1) 两类。有害概率超过 0.5 则判为有害，任一模型判有害 → 丢弃。

③ 流畅度检测 (PerplexityScorer)

目的：检测文本是否自然流畅，排除语病或乱序内容。

- 使用本地 Qwen1.5-1.8B 模型计算文本的困惑度 (PPL)
- PPL 低 → 文本自然、通顺；PPL 高 → 文本生硬、杂乱
- 若 PPL 超过设定阈值 (默认 80) → 丢弃

程序流程

步骤	说明
1	导入依赖：加载 torch、transformers、tqdm 以及正则处理相关库。

2	读取原始数据：逐行加载中文样本，支持普通文本或 messages 结构。
3	规则过滤：使用正则表达式过滤空白、超长、超短、网址、邮箱和长数字串等问题文本。
4	加载安全模型：分别加载多语言毒性分类模型和中文风险检测模型。
5	执行有害检测：对每条文本进行两轮安全筛查，过滤不安全内容。
6	加载语言模型：使用本地 Qwen 模型作为困惑度评分器。
7	计算困惑度：对文本计算 PPL，过滤表达不自然或质量较差的内容。
8	保存结果：把通过全部检测的样本写出为清洗后的数据文件。
9	输出统计：打印规则过滤、安全过滤和困惑度过滤后的保留情况。

输出报告示例

```
清洗报告：
rule:  200 条过滤
tox1:   50 条过滤
tox2:   30 条过滤
ppl:  120 条过滤
保留样本：4600 / 5000
输出文件： ./cleaned_chinese_data_offline.jsonl
```

适用场景： 在离线环境下对中文训练语料进行质量控制、安全过滤和流畅度筛选。